AIM:-

Write a program to implement error detection and correction using HAMMING code concept. Make a test run to input data stream and verify error correction feature.

Create sender program with below features:

1. Input to sender file should be a text of any length program should convert the text to binary
2. Apply hamming code concept on the binary data and add redundant bits to it.
3. Save o/p in a file called channel.

Create a Receiver program with below features.

1. Receiver program should read the i/p from channel file.
2. Apply hamming code on the binary data to check for errors.
3. If there is an error, display the position of the error.

```
#.... SENDER
data = input("Enter the data bits:")
m = len(data)
# find no. of parity bits
r = 0
while 2**r < m+r+1:
    r += 1
# create hamming array
hamming = [0] * (m+r+1)
# Place data bits.
i = 0
for i in range(1, m+r+1):
    if i & (i-1) != 0:
        hamming[i] = int(data[j])
        j += 1
# Calculate parity bits
for i in range(r):
    pos = 2**i
    parity = 0
    for j in range(1, m+r+1):
        if j & pos:
            parity = hamming[j]
    hamming[pos] = parity
# Display hamming code in reverse order
print("Hamming code:", end=" ")
for i in range(m+r, 0, -1):
```

```
print(hamming[i], end=" ")
for i in range(n+r, 0, -1):
print()
```

- RECEIVER

```
received = input("enter the recieved
           hamming code:")
n = len(received)
# create hamming array hamming
hamming = [0]*(n+1)
for i in range(1, n+1):
    hamming[i] = int(received[n-i])
r = 0
while 2**r < n+1:
    r += 1
# find error position.
    error_position = 0
for i in range(r):
    pos = 2**i
    parity = 0
for j in range(1, n+1):
    if j & pos:
    Parity = hamming[j]
error_position += parity * pos
# check error
    if error_position != 0:
```

```
print("Error detected at position:", error_position)
# Correct the error
hamming[error_position] ^= 1
else:
    print("\nNo error detected:")
# Display corrected hamming code in reverse order
print("corrected hamming code:", end=" ")
for i in range(n,0,-1):
    print(hamming[i], end=" ")
print()
# extract original data
data=""
for i in range(1, n+1):
    if i (i-1)==0:
    data += str(hamming[i])
print("original data:", data).
```

Result:-

Thus the study of implement error detection and correction using HAMMING CODE. compiler make a test run to I/P data strategy and verify error connection is done.